algorithms

MDPI

Article

Efficient Record Linkage in the Age of Large Language Models: The Critical Role of Blocking

Nidhibahen Shah [1], Sreevar Patiyara [2], Joyanta Basak [1], Sartaj Sahni [3], Anup Mathur [4], Krista Park [4] and Sanguthevar Rajasekaran [1,*]

[1] School of Computing, University of Connecticut, 371 Fairfield Way, Storrs, CT 06269, USA; nidhibahen.shah@uconn.edu (N.S.)
[2] CS Department, Purdue University, 305 N. University St., West Lafayette, IN 47907, USA
[3] CISE Department, University of Florida, Gainesville, FL 32611, USA
[4] U.S. Census Bureau, 4600 Silver Hill Road, Washington, DC 20233, USA
* Correspondence: sanguthevar.rajasekaran@uconn.edu

Abstract

Record linkage is an essential task in data integration in the fields of healthcare, law enforcement, fraud detection, transportation, biology, and supply chain management. The problem of record linkage is to cluster records from various sources such that each cluster belongs to a single entity. Scalability in record linking is limited by the large number of pairwise comparisons required. Blocking addresses this challenge by partitioning data into smaller parts, substantially reducing the computational cost. With the advancement of Large Language Models (LLMs), there are several possibilities to improve record linkage by leveraging their semantic understanding of textual attributes. LLM-based record linkage algorithms in the literature have very large runtimes. In this paper, we show that the employment of blocking can result in significant improvements not only in the runtime but also in the accuracy. Specifically, we propose a record linkage algorithm that combines LLMs with blocking. Experimental evaluation demonstrates that our algorithm achieves lower runtimes while simultaneously improving F1 scores compared to the approaches relying solely on LLMs. These findings demonstrate the importance of blocking even in the era of advanced machine learning models.

Keywords: record linkage; blocking; large language models

check for updates

Academic Editor: Edward Rolando Núñez-Valdez

Received: 30 September 2025
Revised: 3 November 2025
Accepted: 11 November 2025
Published: 16 November 2025

Citation: Shah, N.; Patiyara, S.; Basak, J.; Sahni, S.; Mathur, A.; Park, K.; Rajasekaran, S. Efficient Record Linkage in the Age of Large Language Models: The Critical Role of Blocking. *Algorithms* **2025**, *18*, 723. https://doi.org/10.3390/a18110723

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Record linkage is a fundamental task in data integration [1]. It refers to the process of clustering the input records in such a way that each cluster of records belongs to one and only one entity. It plays a crucial role in the fields of healthcare, E-commerce, transportation, law enforcement, and biology. In a typical application of interest, there will be multiple datasets where each dataset has records pertaining to entities. A record can be thought of as a set of attributes. Possible attributes include first name, last name, age, gender, address, Social Security Number, and date of birth. For example, in the healthcare domain, each provider has a dataset of records belonging to individuals that they cater to. Our approach leverages the structural specificity of domains like healthcare, where attributes such as patient name and date of birth facilitate effective block information.

The record linkage problem is challenging due to various factors such as the absence of a global key across different datasets, missing values, presence of errors of different kinds, schema and granularity, the use of different formats, and longer runtimes. However,

one of the central challenges of record linkage is scalability. As data increases in size and complexity, the number of record pair comparisons performed by the brute force algorithm increases quadratically. To address this challenge, blocking has been introduced. Blocking partitions data into smaller parts based on shared characteristics. Pairwise record comparisons are performed only within the individual parts. Over the years, various blocking strategies have been introduced. In our prior work, we have introduced Soundex blocking [2] and Double Metaphone Blocking [3] techniques.

The advancements in Large Language Models (LLMs) [4] have opened new opportunities for record linkage. With the ability to capture contextual relationships and semantic meaning in texts, LLMs [5] are well suited for identifying potential matches across databases [6,7]. Related work includes studies like HomLLM [8], which leverages semantic homology relationships for fine-grained bird image classification using Large Language Models, HPCTrans [9], which employs heterogeneous plumage cue-aware texton correlation representation for fine-grained bird image classification via transformers, and DSR-Net [10], which uses distinct selective rollback queries for road crack detection with detection transformers. However, the computational demands of LLMs are non-trivial and applying them to all possible record pairs, especially in large datasets, is impractical and a time-consuming process. To overcome this challenge, we have integrated blocking techniques with LLMs to improve the performance of linking records across datasets. Our method uniquely combines blocking techniques with LLMs to achieve significant scalability and efficiency gain, enabling the processing of large-scale datasets with reduced computational overhead compared to traditional methods. This paper establishes that blocking remains a critical enabler of efficient record linkage in the era of LLMs.

2. Related Work

The literature on record linkage categorizes algorithms into two types: (1) deterministic and (2) probabilistic [11]. The primary distinction lies in how the distance is computed between the record pairs—deterministically using fixed rules or probabilistically. A classical probabilistic algorithm was introduced by Fellegi and Sunter [12]. They assumed independence among record pairs but do not incorporate transitivity or other global constraints. For instance, consider two records $R_1 = a_1^1, a_2^1, \ldots, a_q^1$ and $R_2 = a_1^2, a_2^2, \ldots, a_q^2$. Here, q is the number of attributes and the a_i^j's are attribute values.

Fellegi and Sunter's [12] algorithm computes the distance d_i between a_i^1 and a_i^2 for every $1 \leq i \leq q$ and composes $d_1, d_2, \ldots, d_q$ to arrive at a distance between R_1 and R_2. The Bayesian approach underpins their probabilistic model with a blocking step to reduce the computational overhead. However, the algorithm can only link two datasets at a time and often struggles with large and complex data.

Sadinle and Fienberg [13] proposed an extension of this model to support multiple datasets, though scalability remains a challenge. In practice, tools such as FastLink [14] which is developed around Fellegi and Sunter's model, have been widely adopted. More recently, SPLINK [15] has also emerged as a faster implementation. However, when applied to modern large-scale datasets, both systems exhibit limitations in accuracy and runtime efficiency.

Deterministic approaches, such as those implemented in FEBRL [16,17], rely on predefined similarity metrics like edit distance, q-gram distance, and Hausdorff distance. FEBRL often under-performs on both accuracy and runtime compared to most recent deterministic algorithms. Despite some improvements, there remains a considerable scope for advancing scalability and accuracy in record linkage. Record linkage has long been recognized as vital in medical and healthcare fields. For example, in 1900, Alexander Graham Bell [18]

combined genealogical and administrative records, such as marriage and census data, to study familial patterns of deafness.

Similarly, in 1929, R.A.Fisher [19] integrated public records with family data for human genetics research. Since then, record linkage has been widely adopted in the biomedical field. In one study, Victor and Mera [20] linked patient and healthcare provider records across time and geographic regions using a mix of exact and probabilistic algorithms. Their dataset was drawn from insurance claim databases and included 52 million records representing over 20 million individuals. Their algorithm involved three key steps: data standardization, weight estimation, and matching. Its effectiveness was validated through divergent, convergent, and criterion validity tests.

The authors of the paper [21] emphasize the shift in healthcare from institution-centered to consumer-centered care, nothing that accurate patient identification is crucial for effective healthcare reform and delivering fast, safe, and high-quality care. They proposed an algorithm to identify exact and approximate duplicate medical identity records. The algorithm has three steps: data standardization, matching similar record pairs, and creating clusters of related records. When applied to a dataset of 300,000 records, it identified 240,000 unique clusters.

Padmanabhan, Carty, and other authors [22] described the record linkage approach used by NHS Digital and the Clinical Practice Research Datalink in England. CPRD routinely links primary care data with various health-related datasets, facilitating comprehensive health research. A significant application of record linkage in biomedicine involves connecting genealogical records with morbidity, mortality, and medical data. The linkage aids in identifying the genetic basis of diseases and understanding how patients with different genetic profiles respond to treatments [23].

Most record linkage algorithms have a quadratic time complexity relative to total number of records across datasets. To address this, indexing techniques have been developed to reduce computational demands by filtering out clearly dissimilar record pairs [24]. Another method, blocking, groups records into potentially overlapping blocks, limiting comparisons to records within the same block. Additionally, filtering techniques identify similar record pairs based on similarity threshold under specific metrics. A recent survey on blocking and filtering methods is available in [25]. A key challenge in record linkage lies in the large number of record pairs to be processed. Blocking reduces the number of record pair comparisons by partitioning records into blocks. Pairwise record comparisons are performed only within the blocks.

Numerous blocking strategies can be found in the literature. The Sorted Neighborhood Method [1] sorts records by a key and only compares those within a sliding window. Canopy clustering [1] uses similarity metrics to create overlapping canopies. Token-based blocking is particularly effective for noisy or short strings.

Phonetic encodings are also widely adopted. Soundex [26], one of the earliest phonetic algorithms, maps names with similar pronunciations to the same code. Soundex was first developed by [26] in 1918 and it was first employed by the United States Census Bureau to categorize names for statistical purposes. According to the National Archives, the Soundex index is a method of coding surnames (last names) based on their pronunciation rather than their spelling. Surnames with similar sounds, such as “SMITH” and “SMYTH”, are assigned the same code, allowing them to be grouped together [26]. The Soundex generates one code for each provided name. However, this approach carries a risk of errors, as phonetically similar names may refer to different entities. In our approach, the probability of such errors is reduced, as blocking and clustering does not rely solely on first- and last-name comparisons. We incorporate other attributes such as date of birth and address which are evaluated using a pairwise matching process using the LLaMA3 and BERT model.

Even if the error occurs during blocking due to similar phonetic codes, records are unlikely to cluster together as other attributes differ significantly. Recently, Soundex blocking [2] has been proposed as a novel candidate generation approach that demonstrates improved runtime and F1 score compared to traditional blocking approaches.

Double Metaphone generates two phonetic codes per string to better capture the alternative pronunciations. The Double Metaphone algorithm, which was introduced by Philips [27], generates two codes per name: primary and secondary and handles names in a better way for variations like "Kline" and "Cline". The Double Metaphone is widely adopted as it works well with different languages and catches more phonetic matches than Soundex. Previous studies have used Double Metaphone for matching names in historical records, indicating that it outperforms older methods in accuracy [28]. Another work applied Double Metaphone to customer data deduplication, proving that it can handle noisy real-world data like typographical errors and abbreviations [29]. Double Metaphone has also been used beyond simple pairwise matching. In healthcare, Double Metaphone is part of systems that link patient records across hospitals, where names might vary due to some transcription errors [30]. It has been used in search systems, tweaking it to rank similar-sounding terms for fuzzy queries [31]. However, these studies typically focus on comparing two records at a time and Double Metaphone is used for measuring similarity only. There is one study that tried to use Double Metaphone for clustering by precomputing codes for all records. However, it did not do well for large-scale datasets due to memory and time costs [32].

Blocking tackles the problem of speed in record linkage by dividing the dataset into small groups or blocks, thereby reducing the number of comparisons required and hence reducing the runtime. Standard blocking uses attributes like first name or zip code to create blocks [12]. Using Double Metaphone for approximate string matching in databases (but only as a preprocessing step, not as a full blocking strategy) was considered in [33].

Ref. [34] applied Double Metaphone to link encrypted records. The codes were used to group data securely. However, the focus was privacy, not speed. Shah introduced Double Metaphone Blocking [3] for record linkage, showing that it achieves higher F1 scores and faster runtimes compared to state-of-the-art blocking methods. The emergence of Large Language Models (LLMs) has opened new opportunities for record linkage. Systems such as DeepMatcher [35] and Ditto [36] demonstrate that pre-trained transformer models significantly outperform classical methods in record linkage tasks.

Recent studies show that fine-tuned LLMs can be generalized across domains. Moreover, generative models are being explored for explainable linkage decisions. However, LLMs face several limitations, such as scalability issues, calibration changes, and performance degradation in noisy or real-world datasets compared to curated benchmarks. To further contextualize our approach, we integrate insights from recent foundational works. Choi et al. [37] propose a multi-stage explainable clustering pipeline with network-based visualization, complementing our focus on interpretable analysis of high-dimensional records.

3. Methodology

Building on our prior works, we recognize that blocking is an important enabler of scalability in the record linkage process. Blocking reduces the number of record pair comparisons. The recent advancement of LLMs provides an opportunity to learn semantic similarity between the records. We have explored the combined impact of blocking and LLMs using the framework shown in Figure 1. Algorithm 1 also provides more details on our framework. The workflow is divided into the following steps.

Algorithm 1: Integrated Blocking-LLM Record Linkage

Input : Dataset D, Training Dataset D_{train}, Attribute Criteria
Output: Set of matched record pairs

```
begin
    /* Train LLM for pair classification                                  */
    Model M ← FineTuneLLM(D_train) ;          // e.g., LLaMA3.1 or BERT,
     instruct-tuned
    /* Generate blocks with attribute-based blocking                      */
    SelectedAttrs ← SelectAttributes(D, Criteria) ;    // e.g., name, address,
     gender, DOB
    Blocks B = {B_1, B_2, ..., B_n} ← BlockData(D, SelectedAttrs);
    ;                  // ⋃_{i=1}^{n} B_i = D, similar attribute values per block
    /* Match pairs within blocks using LLM                                */
    MatchedPairs ← ∅;
    foreach block B_i in B do
        Pairs ← GeneratePairs(B_i);
        foreach pair (r_a, r_b) in Pairs do
            Prediction ← Classify(M, (r_a, r_b));
            if Prediction = match then
                MatchedPairs ← MatchedPairs ∪ {(r_a, r_b)};
            end
        end
    end
    return MatchedPairs;
end
```

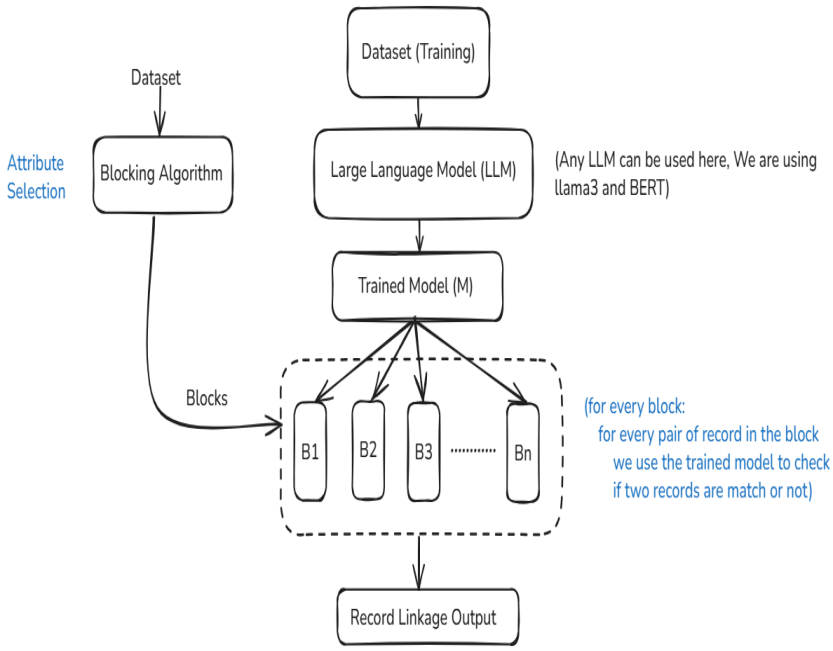

Figure 1. Workflow for record linkage using blocking and LLMs.

3.1. Dataset Preparation

The process begins with the selection of a dataset that has to be linked. In our experiments, we use six different datasets. All these datasets contain different attributes.

Each dataset varies in size, structure, and attribute composition. Each dataset contains a unique set of identifying attributes such as names, address, date of birth, and other demographic fields. Prior to training, we standardize the data formats and normalize string representations such as converting to lower case, handling the missing values. This preprocessing step ensures consistency across datasets.

3.2. Blocking Algorithm

Not all the attributes are important for the linkage task. Therefore, using an attribute selection algorithm [38], we choose a subset of the attributes for blocking. Attributes such as name, address, gender, and date of birth get selected often as they carry the most semantic weight in determining whether two records match or not. Based on the selected attributes, the blocks are generated. We employ a k-mer-based attribute selection algorithm as described in [38]. This algorithm is critical for reducing the comparison space in record linkage by grouping similar records into blocks based on selected attributes, such as name or address. The algorithm operates in two stages: deduplication and k-mer-based blocking. First, the input dataset is deduplicated to remove identical records, ensuring computational efficiency by avoiding redundant record pair comparisons. Next, for the chosen blocking attribute (e.g., name), the algorithm generates k-mer sub strings of length k. Each k-mer serves as a blocking key, and records are grouped into blocks if they share the same k-mer. Each block contains records that share similar attribute values. Formally, let D denote the dataset and f be the blocking function. The dataset is partitioned into blocks:

$$B = \{B_1, B_2, \ldots, B_n\}, \bigcup_{i=0}^{n} B_i = D \tag{1}$$

Each B_i is then processed independently, thereby improving the scalability while maintaining a high likelihood of retaining the true matches.

3.3. LLM Training

In parallel with blocking, we use a training dataset to train the LLM models. Here, we use the LLaMA3.1 8B Instruct model, [39] which has 8 billion parameters, and the BERT-based uncased model [40], which has 110 million parameters, for training. LLaMA3.1 is a state-of-the-art instruction-tuned model known for its strong contextual reasoning ability. BERT is a bidirectional transformer pre-trained on large text repositories widely used for entity matching. Related work includes works such as LDCNet [41], which focuses on limb direction cue-aware networks for adaptable human pose estimation in industrial behavioral biometrics systems, EHPE [42], which utilizes skeleton cue-based Gaussian coordinate encoding for efficient human pose estimation, and TransIFC [43], which employs invariant cue-aware feature concentration learning for efficient fine-grained bird image classification. Both models are fine-tuned to classify whether a given pair of record is a match or non-match. The training data consists of record pairs labeled according to the ground truth linkages. Once the training is completed, we obtain the trained model M. The separate models are fine-tuned for each dataset in our experiments. Here are the fine-tuning details for the Llama3 model:

- Model loading: the base model is loaded from "unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit" with quantization to reduce memory usage. The maximum sequence length is set to 2048 tokens.
- Parameter Efficient Fine-Tuning setup: LoRA(Low Rank Adaptation) is applied with rank (r = 16), alpha = 16, and dropout = 0.

- Training Configuration: Uses Supervised Fine-Tuning trainer from TRL library. Per device batch size = 2. Optimizer "adamw8bit" with learning rate = 2×10^{-4}, weight decay = 0.01.

  Below are the fine-tuning details for the BERT model.
- Tokenizer and Model loading: tokenizer and model loaded from "bert-base-uncased" with number of labels = 2
- Training Configuration: batch size per device = 4, gradient accumulation steps = 8, learning rate = 2×10^{-5}, weight decay = 0.01.

3.4. Pairwise Matching Within the Blocks

Once the LLM-based model M is trained, it is applied to each block. For each block B_i, the model evaluates all possible record pairs within that block $(r_a, r_b) \in B_i$. The trained model outputs if it is a match or not. This step reduces the number of record pair comparisons as the comparison is performed within each block and not the whole dataset. By performing the pairwise comparisons within blocks rather than across the entire dataset, this approach significantly reduces computational complexity while maintaining high recall. The blocking step ensures that only records with a high likelihood of referring to the same entity are compared. This minimizes the redundant or irrelevant comparisons. Furthermore, the model's predictions allow us to capture subtle semantic similarities between attributes.

3.5. Record Linkage Output

Finally, the decision from all blocks are aggregated into record linkage output. This output identifies all record pairs that the model classified as matches. For graph construction, each positive pair forms an edge in the undirected graph, with records as nodes. Cluster inference then applies transitive closure to connect all nodes linked by paths of matches. As a simple example, given records $R1$, $R2$, and $R3$, if $R1 - R2$ is a match and $R2 - R3$ is a match, then the final cluster is $R1, R2, R3$.

4. Experimental Setup

We conducted our experiments using two computing environments:

- NVIDIA TITAN RTX GPU with an AMD Ryzen Threadripper 2950X 16-Core Processor. All experiments with Llama3.1 were performed on this environment.
- NVIDIA TITAN XP GPU (11.896 GB memory) with an Intel(R) Core(TM) i5-8400 CPU @ 2.80GHz. BERT experiments were performed with this environment.

These configurations enabled us to efficiently train and evaluate LLaMA3 and BERT across multiple datasets.

4.1. Datasets

To evaluate the robustness of our proposed algorithm, we conducted experiments with six datasets: FEBRL, DS, OCR, Phonetic, Typo, and North Carolina Voter Dataset. These datasets vary in size, structure, and types of noise, which allows us to test the generalization of our approach.

- FEBRL (Freely Extensible Biomedical Record Linkage): FEBRL [17] is perhaps the most widely used dataset for record linkage tasks. It consists of synthetically generated records, with introduction of errors such as misspellings, missing values, and typographical variations to simulate real-world scenarios. There are multiple versions of FEBRL available, such as febrl-1 through febrl-4. Each of them varies in size and error distribution. In our experiment, we used the febrl-4 dataset provided by recordlinkage

package in Python. This contains two datasets to be linked along with the ground truth mapping of true matches.The febrl-4 dataset comprises 10,000 records in total: 5000 original records and 5000 duplicates, with exactly one duplicate per original to provide a controlled 1:1 matching structure and ground truth labels for evaluation. The attributes include first name, last name, address, suburb, date of birth, and Social Security ID. The duplicates incorporate realistic error types to mimic the data quality issues in real-world datasets. We selected febrl-4 over other versions like febrl-1 with 1000 records or febrl-3 with 3000 records due to its large-scale controlled duplicate structure.

- DS Dataset: To evaluate our algorithms, we utilized real-world data sourced from the Social Security Death Master File, provided by SSDMF.INFO [44]. Each record includes the following attributes: Social Security Number, last name, first name, date of birth, and date of death. To simulate realistic errors, we employed a modified version of the FEBRL dataset generator program to introduce variations into the data.
- OCR (Pseudopeople Dataset): This dataset is generated from the Pseudopeople library from Census, which generates large-scale simulated populations. It contains slightly over 10 million records of people in the state of Michigan. In this variant, 10% OCR errors were introduced in the first name and last name attributes, simulating character recognition mistakes.
- Phonetic (Pseudopeople Dataset): Similarly generated from the Pseudopeople library, this dataset introduces 10% phonetic errors in the first name and last name fields. These errors capture common variations in spellings due to phonetic similarities such as Smith vs Smyth. This makes it necessary to evaluate phonetic-based records linkage approaches.
- Typo (Pseudopeople Dataset): This dataset introduces 10% typographical errors in the first name and last name attributes. These types of errors are commonly observed in administrative databases.
- North Carolina Voter Dataset (NCV): The NCV [45] dataset contains five files, each containing one million voter records. Each file is source-consistent, meaning that within each file, there is only one record per entity. However, individuals may appear across multiple files. Some entities may be present in all the five files, others in two or three files, and some in only one file. This dataset is used in multi-source record linkage.

4.2. Evaluation Metrics

This study employs two primary metrics to evaluate performance of the proposed method. The first metric is the F1 score, a widely used measure for assessing the accuracy of binary classification tasks, such as record linkage. The F1 score combines precision and recall into a single value, providing a balanced evaluation of the model's ability to correctly identify true positive matches while minimizing both false positives and false negatives. A higher F1 score indicates a better performance. Mathematically, the F1 score is defined as follows:

$$F1 = 2 * (Precison \times Recall)/(Precision + Recall) \tag{2}$$

The second metric is runtime, which is particularly important in the context of record linkage involving large datasets. Runtime reflects the computational efficiency of the method, with lower runtime indicating faster and more efficient processing.

4.3. Implementation Detail

Our implementation was developed in Python 3.10 on Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-153-generic X86_64). The following key libraries and frameworks were used:

- PyTorch with CUDA 12.2 for LLM training and interface.
- Transformers (Hugging Face) for loading and fine-tuning the LLaMA3 model.
- Unsloth library for efficient model loading and handling.
- tqdm for progress tracking and runtime monitoring.
- recordlinkage for dataset preparation and benchmark record linkage tasks.
- scikit-learn for computing evaluation metrics.

The LLaMA3 model was trained on the above datasets. Blocking was applied using our attribute-based blocking function. It generates candidate record pairs within each block. These pairs were formatted as textual sequences and classified using the trained LLaMA3 model. The model leveraged a context length of 2048 tokens with 4-bit quantization via Unsloth to reduce memory footprint. Quantization has several potential impacts on performance. It lowers memory usage and speeds up computation by using fewer bits to represent weights and activation, which is great for efficiency, especially on resource-limited devices. In parallel, a BERT-based uncased model was trained for sequence classification. Training data was first transformed into instruct-input-output format, where each record pair was wrapped as: Instruction: "Determine if these records match:", Input: "<record pair text>", Output: "<label:match / not a match>". The dataset was tokenized using BERT's tokenizer with truncation and padding, and labels were mapped to binary classes. Fine-tuning was performed using the Hugging Face Trainer API with the following configurations:

- Batch size: 4 per device
- Gradient accumulation: 8 steps
- learning rate: 2×10^{-5}
- Number of epochs: 1
- Mixed-precision training: fp16
- Weight decay: 0.01

The hyperparameters listed were selected based on a combination of standard practices from the literature. Specifically, values such as learning rate 2×10^{-5}, batch size 4, and number of epochs 1 align with recommended settings for fine-tuning transformer models like BERT and LLaMA3 as documented in Hugging Face guidelines. The 8 steps of gradient accumulation and mixed precision training fp16 were chosen to optimize memory usage and training stability. The evaluation was performed using standard classification metrics like precision, accuracy, recall, and F1 score. After training, the model tokenizers were saved for inference. For both LLaMA3 and BERT, the inference process follows a common pipeline. First, we preprocess the dataset by performing deduplication, blocking, and candidate pair generation. Each candidate pair within the block is then passed to the model in the textual form, where the model evaluates whether two records represent the same entity. In the case of BERT, this is achieved through a sequence classification head that outputs the probability of match and non-match. This means at the test time, we tokenize the candidate pair, feed it to the fine-tuned BERT model, and use the output probabilities to decide if it is a match. For LLaMA3, we use an instruction-tuned prompting setup, where the model is asked to determine if two sequences correspond to the same entity. Regardless of the model, the outputs are converted into binary decisions, which are then used to identify the linked records. Finally, the predictions are compared against the ground truth to compute the evaluation metrics.

5. Results and Discussion

The experiments demonstrate that our approach results in a substantial improvement in F1 scores when we incorporate blocking with LLaMA3 and BERT models for record linkage tasks across six different datasets. This is shown in Table 1. We maintained the total number of record pair comparisons at approximately 2 million for each dataset. For

LLaMA3, blocking elevated F1 scores significantly. For the FEBRL data, F1 score increases from 91.24% to 96.03%; for DS data, the increase is from 77.65% to 88.48%; for OCR data, the F1 score improves drastically from 9.20% to 94.28%. For phonetic and Typo datasets the improvements are from 16.03% to 93.39% and from 38.02% to 94.24%, respectively. For NCV data, the F1 score improves slightly from 93.29% to 99.73%. With BERT, we also saw similar improvements as shown in Table 2. These results highlight the effectiveness of blocking in boosting the accuracy of LLM-based record linkage.

Table 1. F1 score comparison for LLaMA3.1 8B Instruct.

Dataset	Blocking	No Blocking
FEBRL	96.03%	91.24%
DS	88.48%	77.65%
OCR	94.28%	9.20%
Phonetic	93.39%	16.03%
Typo	94.24%	38.02%
NCV	99.73%	93.29%

Table 2. F1 score comparison for BERT-based uncased.

Dataset	Blocking	No Blocking
FEBRL	100.00%	100.00%
DS	89.57%	100.00%
OCR	94.12%	13.63%
Phonetic	93.91%	22.67%
Typo	94.12%	45.09%
NCV	98.32%	50.23%

For the DS dataset in BERT, we can see that the F1 score with blocking is lower than non-blocking. These variation stem from dataset specific characteristics and trade-offs inherent in the blocking approach, which balances efficiency with recall. For the DS dataset, the drop in F1 score with blocking indicates a minor recall loss due to the partitioning. On the other hand, for NCV, the improvement with blocking highlights precision gain in noisy data.

A significant achievement of the study is the remarkable reductions in the runtimes as show in Tables 3 and 4 that we obtain through blocking. Our methodology enhances the practicality of LLM applications. For LLaMA3, the runtime decreased dramatically. For FEBRL, the runtime drops from 18:13:53 h to 1:24:10; for OCR, from 73:52:18 h to 0:05:35 min, and for phonetic, from 73:52:18 h to 0:06:19 min. For NCV, the runtime reduces from 70:51:55 h to 0:19:02 min. BERT also benefited from blocking. For FEBRL data, the runtime reduces from 7:37:02 h to 0:40:10 min. For OCR data, the runtime reduces from 39:51:00 h to 0:02:52 min. For phonetic data, the time reduces from 39:44:20 h to 0:03:19 min. For typo data, the reduction is from 39:14:08 h to 0:03:02 min and for the NCV data, the runtime reduces from 21:21:05 h to 0:05:49 min.

Figure 2 visualizes the improvements in F1 scores. The log-scale chart in Figure 3 displays the reductions in runtimes corresponding to the six different datasets for which experiments were performed. These experiments demonstrate that using blocking with LLM not only maintains high F1 scores but also significantly improves the runtime performance. Thus, blocking is indispensable in the context of large-scale record linkage tasks.

Table 3. Runtime comparison for LLaMA3.1 8B Instruct.

Dataset	Blocking (Hours)	No Blocking (Hours)
FEBRL	1:24:10	18:13:53
DS	0:07:37	18:14:09
OCR	0:05:35	73:58:48
Phonetic	0:06:19	73:52:18
Typo	0:05:31	74:38:55
NCV	0:19:02	70:51:55

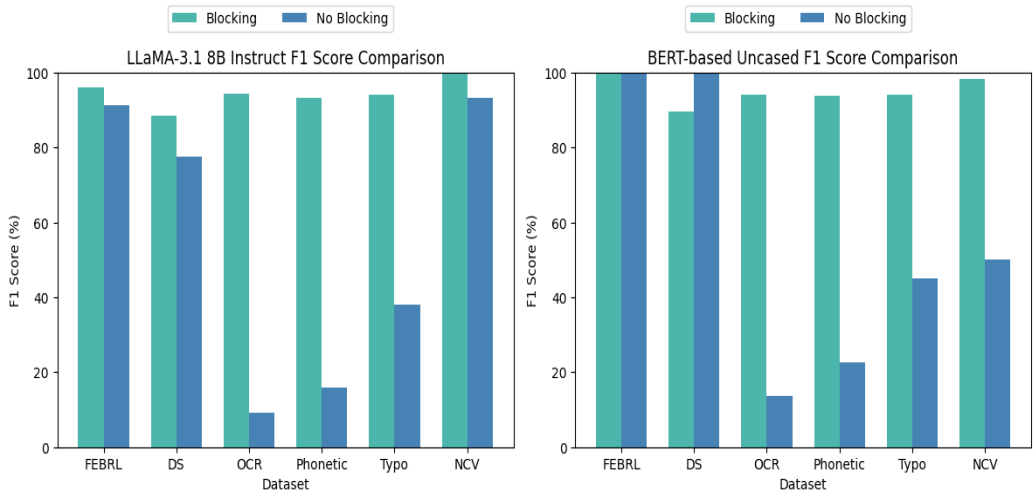

Figure 2. F1 score comparison for LLaMA3 and BERT.

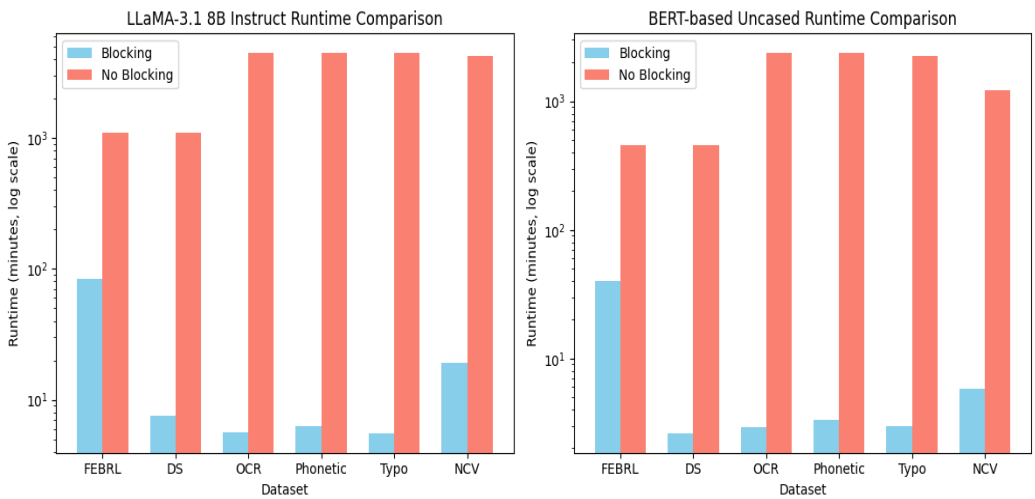

Figure 3. Runtime comparison for LLaMA3 and BERT.

Table 4. Runtime comparison for BERT-based uncased.

Dataset	Blocking (Hours)	No Blocking (Hours)
FEBRL	0:40:10	7:37:02
DS	0:02:35	7:37:02
OCR	0:02:52	39:51:00
Phonetic	0:03:19	39:44:20
Typo	0:03:02	39:14:08
NCV	0:05:49	21:21:05

6. Conclusions and Future Work

The research conclusively demonstrates that integrating blocking with LLMs significantly enhances both F1 scores and runtime efficiency in the record linkage tasks across diverse datasets. These results reveal substantial improvements in accuracy when blocking is employed. Also, there is a remarkable reduction in processing times. This dual benefit indicates the robustness of the blocking approach, making it a valuable technique for improving LLMs' performance in record linkage applications.

For future work, integration of blocking with other advanced LLM models can be investigated. Various blocking strategies can be explored with LLMs. This could further optimize the performance. To further enhance the contribution of this study, we plan to extend our work in several directions. We aim to validate our method on large-scale, real-world datasets, such as enterprise CRM [46] and healthcare EHR [47] systems. Additionally, we will explore advanced blocking strategies, such as Soundex blocking, Double Metaphone Blocking, and super blocking, to evaluate their impact on efficiency and scalability in record linkage tasks. To provide a more comprehensive evaluation of our approach, we plan to incorporate comparisons with state-of-the-art record linkage systems like Ditto and DeepMatcher.

Author Contributions: Conceptualization, N.S., S.R.; methodology, N.S., S.S., S.R.; software, N.S., S.P.; validation, N.S., S.P., J.B., S.S., A.M., K.P., S.R.; formal analysis, S.R., S.S.; investigation, S.R., S.S.; resources, A.M., K.P.; data curation, N.S., S.P., J.B.; writing—original draft preparation, N.S., S.P., S.R.; writing—review and editing, N.S., S.R., A.M., K.P.; visualization, N.S.; supervision, S.R., S.S.; project administration, A.M., K.P.; funding acquisition, S.R. All authors have read and agreed to the published version of the manuscript.

Funding: This work was partially supported by the United States Census Bureau under Award Number CB21RMD0160003. The content is solely the responsibility of the authors and does not necessarily represent the official views of the US Census Bureau.

Data Availability Statement: The code and datasets used in this study are available in the GitHub repository https://github.com/Nidhibahenshah/LLM-with-Blocking (accessed on 24 September 2025).

Acknowledgments: The authors would like to extend their heartfelt gratitude to Kenneth Haase, Daniel Weinberg, Rebecca Steorts, Haley S. Hunter-Zinck, Wendy L. Martinez, and Jennifer Hutnick for their invaluable discussions and constructive feedback, which greatly enriched this research.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

LLM	Large Language Model
FEBRL	Freely Extensible Biomedical Record Linkage
NCV	North Carolina Voter dataset

References

1. Papadakis, G.; Ioannou, E.; Thanos, E.; Palpanas, T. *Four Generations of Entity Resolution*; Springer: Berlin/Heidelberg, Germany, 2021.
2. Shah, N.; Soliman, A.; Basak, J.; Sahni, S.; Haase, K.; Mathur, A.; Park, K.; Weinberg, D.; White, J.; Rajasekaran, S. The Soundex Blocking: A Novel Blocking Approach for Record Linkage. In Proceedings of the 2024 IEEE International Conference on Big Data (BigData), Washington, DC, USA, 15–18 December 2024; pp. 4039–4047. [CrossRef]

3. Shah, N.; Basak, J.; Sahni, S.; Mathur, A.; Park, K.; Weinberg, D.; Rajasekaran, S. Double Metaphone Blocking: An Innovative Blocking Approach to Record Linkage. In *International Symposium on Bioinformatics Research and Applications*; Springer Nature: Singapore, 2025; pp. 139–150.
4. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention Is All You Need. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 5998–6008.
5. Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I. Improving Language Understanding by Generative Pre-Training. In Proceedings of the 2018 OpenAI Workshop, San Francisco, CA, USA, 24 September 2018; pp. 1–12.
6. Chu, Z.; Ni, S.; Wang, Z.; Feng, X.; Li, C.; Hu, X.; Xu, R.; Yang, M.; Zhang, W. History, Development, and Principles of Large Language Models-An Introductory Survey. *arXiv* **2024**, arXiv:2402.06853. [CrossRef]
7. Xhst. Unstructured Record Linkage Using Siamese Networks and Large Language Models (LLMs). Available online: https://github.com/Xhst/ml-record-linkage (accessed on 24 September 2025).
8. Liu, M.; Roy, S.; Li, W.; Zhong, Z.; Sebe, N.; Ricci, E. Democratizing Fine-grained Visual Recognition with Large Language Models. In Proceedings of the Twelfth International Conference on Learning Representations, Vienna, Austria, 7–11 May 2024. Available online: https://openreview.net/forum?id=c7DND1iIgb (accessed on 8 October 2025).
9. Liu, H.; Zeng, S.; Deng, L.; Liu, T.; Liu, X.; Zhang, Z.; Li, Y.-F. HPCTrans: Heterogeneous Plumage Cues-Aware Texton Correlation Representation for FBIC via Transformers. *IEEE Trans. Circuits Syst. Video Technol.* 2025, *in press*. [CrossRef]
10. Deng, Y.; Ma, J.; Wu, Z.; Wang, W.; Liu, H. DSR-Net: Distinct Selective Rollback Queries for Road Cracks Detection with Detection Transformer. *Digit. Signal Process.* **2025**, *164*, 105266. [CrossRef]
11. Enamorado, T.; Fifield, B.; Imai, K. Using a Probabilistic Model to Assist Merging of Large-Scale Administrative Records. *Am. Polit. Sci. Rev.* **2019**, *113*, 404–422. [CrossRef]
12. Fellegi, I.P.; Sunter, A.B. A Theory for Record Linkage. *J. Am. Stat. Assoc.* **1969**, *64*, 1183–1210. [CrossRef]
13. Sadinle, M.; Fienberg, S. A Generalized Fellegi-Sunter Framework for Multiple Record Linkage with Application to Homicide Record Systems. *J. Am. Stat. Assoc.* **2013**, *108*, 385–397. [CrossRef]
14. Enamorado, T.; Fifield, B.; Imai, K. *FastLink: Fast Probabilistic Record Linkage with Missing Data. R Package, Version 0.6.1*; CRAN: Vienna, Austria, 2019. Available online: https://CRAN.R-project.org/package=fastLink (accessed on 24 September 2025).
15. Ministry of Justice (MoJ). *Splink: MoJ's Open Source Library for Probabilistic Record Linkage at Scale. Version 1.0*; GitHub: London, UK, 2021. Available online: https://github.com/moj-analytical-services/splink (accessed on 24 September 2025).
16. Christen, P.; Churches, T. FEBRL—Freely Extensible Biomedical Record Linkage. In *Joint Computer Science Technical Report Series (Online)*; TRCS-02-05; Australian National University, Department of Computer Science: Canberra, Australia, 2002.
17. FEBRL. Available online: http://sourceforge.net/projects/febrl/ (accessed on 24 September 2025).
18. Bell, A.G. The Deaf. In *Special Reports: The Blind and the Deaf*; U.S. Department of Commerce and Labor, Bureau of the Census, Eds.; U.S. Government Printing Office: Washington, DC, USA, 1900.
19. Box, J.F. *R.A. Fisher, the Life of a Scientist*; Wiley: New York, NY, USA, 1978.
20. Victor, T.W.; Mera, R.M. Record Linkage of Healthcare Insurance Claims. *Stud. Health Technol. Inform.* **2001**, *84*, 1409–1413.
21. Sauleau, E.A.; Paumier, J.P.; Buemi, A. Medical Record Linkage in Health Information Systems by Approximate String Matching and Clustering. *BMC Med. Inform. Decis. Mak.* **2005**, *5*, 32. [CrossRef]
22. Padmanabhan, S.; Carty, L.; Cameron, E.; Ghosh, R.E.; Williams, R.; Strongman, H. Approach to Record Linkage of Primary Care Data from Clinical Practice Research Datalink to Other Health-Related Patient Data: Overview and Implications. *Eur. J. Epidemiol.* **2019**, *34*, 91–99. [CrossRef]
23. Kim, D.; Labkoff, S.; Holliday, S.H. Opportunities for Electronic Health Record Data to Support Business Functions in the Pharmaceutical Industry—A Case Study from Pfizer, Inc. *J. Am. Med. Inform. Assoc.* **2008**, *15*, 581–584. [CrossRef]
24. Christen, P. A Survey of Indexing Techniques for Scalable Record Linkage and Deduplication. *IEEE Trans. Knowl. Data Eng.* **2012**, *24*, 1537–1555. [CrossRef]
25. Papadakis, G.; Skoutas, D.; Thanos, E.; Palpanas, T. Blocking and Filtering Techniques for Entity Resolution: A Survey. *ACM Comput. Surv.* **2020**, *53*, 31. [CrossRef]
26. Odell, M.; Russell, R. The Soundex Coding System. U.S. Patent US1261167A, 9 April 1918; p. 9.
27. Philips, L. The Double Metaphone Search Algorithm. *C/C++ Users J.* **2000**, *18*, 38–43.
28. Christen, P. A Comparison of Phonetic Encoding Algorithms for Historical Name Matching. In Proceedings of the 15th ACM International Conference on Information and Knowledge Management (CIKM), Arlington, VA, USA, 6–11 November 2006; Association for Computing Machinery: New York, NY, USA, 2006; pp. 123–130.
29. Talburt, J.R.; Zhou, Y. Entity Resolution Using Double Metaphone in Commercial Datasets. In Proceedings of the IEEE International Conference on Information Reuse and Integration (IRI), Las Vegas, NV, USA, 4–6 August 2010; pp. 89–94.
30. Ong, T.C.; Mannino, M.V.; Schilling, L.M. Improving Record Linkage with Phonetic Algorithms in Healthcare. *J. Biomed. Inform.* **2014**, *48*, 45–53.

31. Behm, A.; Ji, S.; Li, C.; Lu, J. Fuzzy Search with Double Metaphone for Approximate Matching. In Proceedings of the 25th IEEE International Conference on Data Engineering (ICDE), Shanghai, China, 29 March–2 April 2009; pp. 456–461.
32. Hassanzadeh, O.; Chiang, F.; Miller, R.J. Clustering Records with Double Metaphone: A Scalability Study. In Proceedings of the 16th International Conference on Database Systems for Advanced Applications (DASFAA), Hong Kong, China, 22–25 April 2011; pp. 201–208.
33. Gravano, L.; Ipeirotis, P.G.; Jagadish, H.V.; Koudas, N.; Muthukrishnan, S.; Srivastava, D. Approximate String Joins with Double Metaphone in Databases. *VLDB J.* **2003**, *12*, 345–364.
34. Karakasidis, A.; Verykios, V.S. Privacy-Preserving Record Linkage Using Phonetic Codes. In Proceedings of the 13th Panhellenic Conference on Informatics (PCI), Corfu, Greece, 10–12 September 2009; pp. 101–106.
35. Mudgal, S.; Li, H.; Ko, T.; Srivastava, A.; Wang, R.; Mitra, S.; Srivatsa, S.; Popa, R.A.; Elmore, A.J.; Halevy, A. Deep Learning for Entity Matching: A Design Space Exploration. In Proceedings of the 2018 International Conference on Management of Data (SIGMOD'18), Houston, TX, USA, 10–15 June 2018; pp. 19–34. [CrossRef]
36. Li, Y.; Li, J.; Suhara, Y.; Doan, A.; Tan, W.-C. Deep Entity Matching with Pre-Trained Language Models. *Proc. VLDB Endow.* **2020**, *14*, 50–58. [CrossRef]
37. Choi, I.; Koh, W.; Koo, B.; Kim, W.C. Network-based exploratory data analysis and explainable three-stage deep clustering for financial customer profiling. *Eng. Appl. Artif. Intell.* **2024**, *128*, 107378. [CrossRef]
38. Deo, N.; Rajasekaran, S.; Kamel, R. Identifying Suitable Attributes for Record Linkage using Association Analysis. In Proceedings of the 2023 IEEE International Conference on Big Data (BigData), Sorrento, Italy, 15–18 December 2023; pp. 5753–5760.
39. Meta AI. *LLaMA 3.1 8B Instruct Model*, Version 3.1; Meta: Menlo Park, CA, USA, 2024. Available online: https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct (accessed on 24 September 2025).
40. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2019**, arXiv:1810.04805. [CrossRef]
41. Liu, T.; Liu, H.; Yang, B.; Zhang, Z. LDCNet: Limb Direction Cues-Aware Network for Flexible Human Pose Estimation in Industrial Behavioral Biometrics Systems. *IEEE Trans. Ind. Informat.* **2023**, *20*, 8068–8078. [CrossRef]
42. Liu, H.; Liu, T.; Chen, Y.; Zhang, Z.; Li, Y.-F. EHPE: Skeleton Cues-Based Gaussian Coordinate Encoding for Efficient Human Pose Estimation. *IEEE Trans. Multimed.* **2022**, *26*, 8464–8475. [CrossRef]
43. Liu, H.; Zhang, C.; Deng, Y.; Xie, B.; Liu, T.; Li, Y.-F. TransIFC: Invariant Cues-Aware Feature Concentration Learning for Efficient Fine-Grained Bird Image Classification. *IEEE Trans. Multimed.* **2023**, *27*, 1677–1690. [CrossRef]
44. SSDMF Homepage. Available online: http://ssdmf.info/download.html (accessed on 24 September 2025).
45. North Carolina State Board of Elections (NCSBE). Voter Registration Data. Available online: https://www.ncsbe.gov/results-data/voter-registration-data (accessed on 24 September 2025).
46. Revinate Engineering. CRM Data Pipeline Record Linkage (Part I). In *Revinate Engineering Blog (Online)*; Revinate: San Francisco, CA, USA, 2016. Available online: https://underthehood.meltwater.com/blog/2020/06/29/the-record-linking-pipeline-for-our-knowledge-graph-part-1/ (accessed on 9 October 2025).
47. Adler-Milstein, J.; Jha, A.K. Health Information Exchange among U.S. Hospitals: Who's In, Who's Out, and What Are the Implications? *Health Aff.* **2017**, *36*, 1420–1428. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.